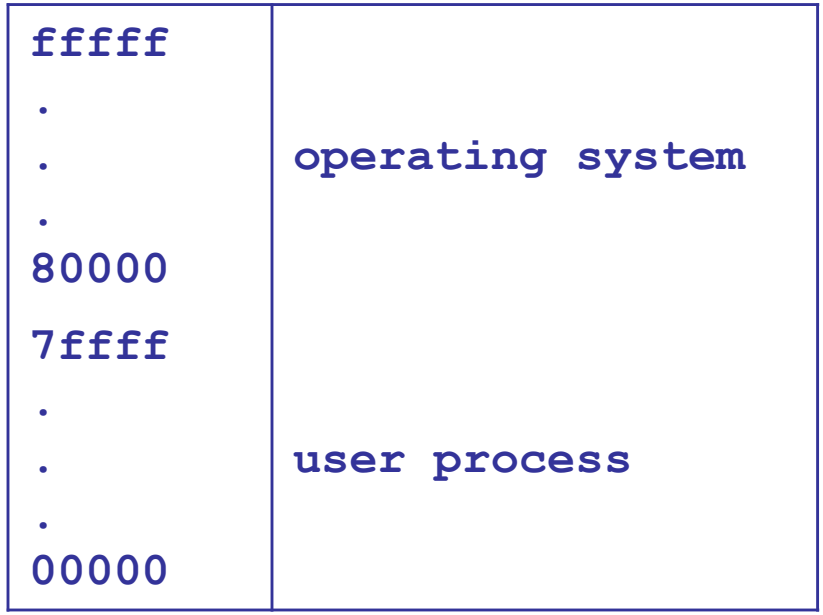
EECS 482
Introduction to operating systems

Dynamic address translation

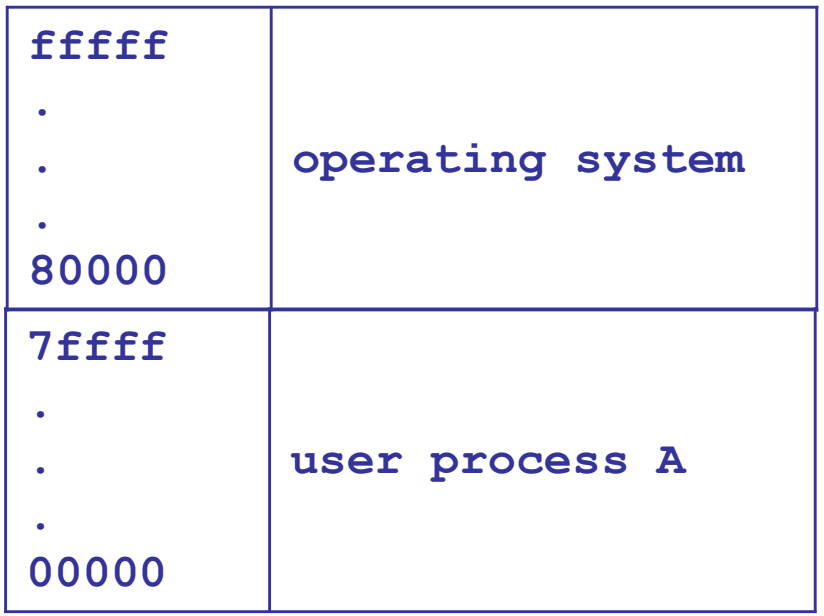
Peter Chen
University of Michigan

Uni-programming

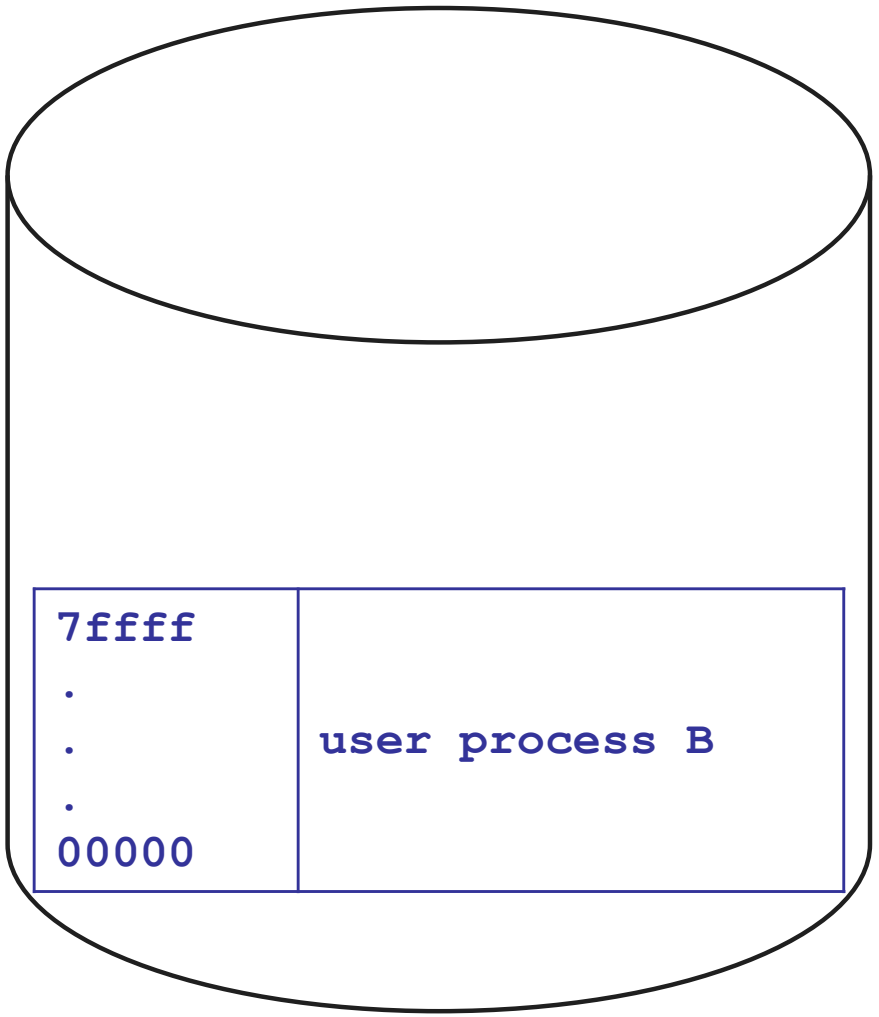
- 1 process occupies memory at a time
- Always load process into same spot in memory
- Reserve space for OS
- Virtual address (process view) = physical address (hardware view)



How to switch processes with uni-programming?



Physical memory



Disk

Uni-programming summary

- Address space abstractions
  - ✓ Address independence?
  - ✓ Protection?
  - ✘ Large address space?

+ Simple (early operating systems used this)

- Expensive context switch

Multi-programming

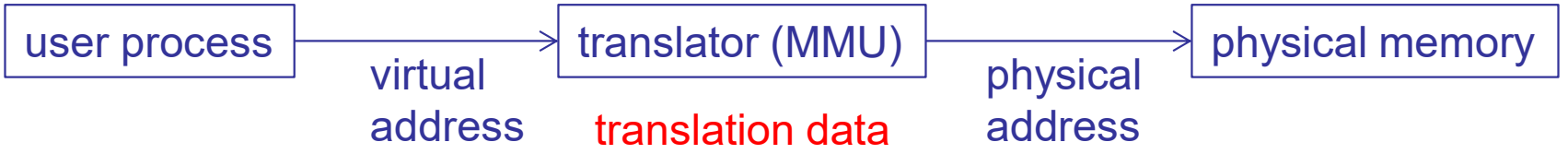
- Allow multiple processes to reside in physical memory at the same time
  - Makes it harder to provide protection
  - Makes it harder to provide address independence

- Providing protection and address independence requires system to do some work on each memory access (!)

Dynamic address translation

- Address independence
  - MMU translates same virtual address in different processes to different physical addresses
- Protection
  - MMU ensures that processes don't access each other's data
- Large address space
  - Virtual address only needs to be in physical memory when accessed

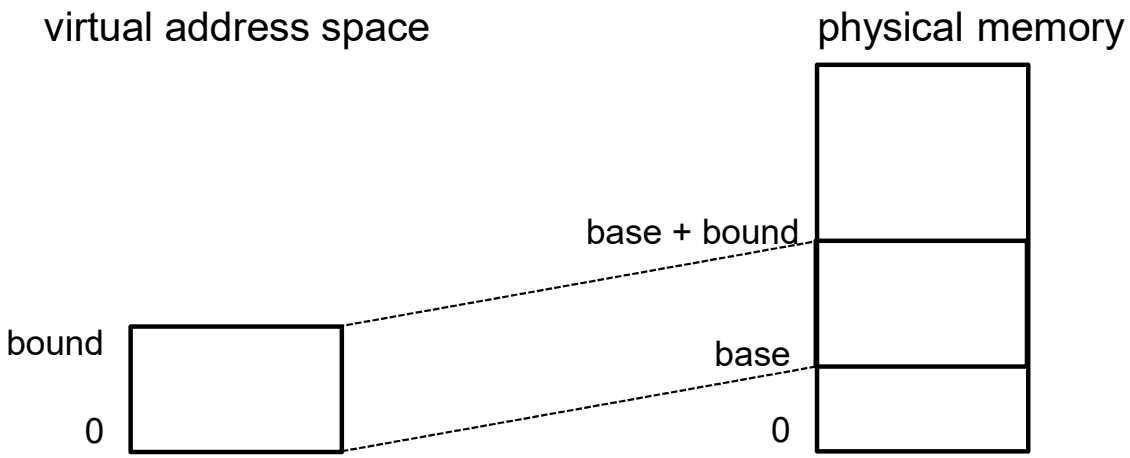
Dynamic address translation



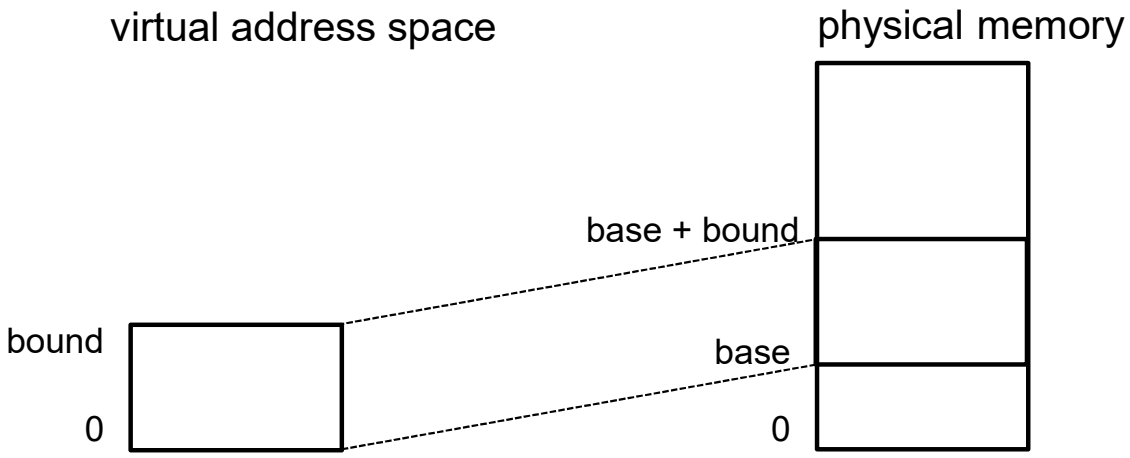
- Translator typically implemented as part of CPU
- Translator uses translation data
  - Each process has its own translation data
  - Changing address spaces = changing data used to translate a virtual address
- Many ways (data structures) to implement translator
  - Speed of translation
  - Size of data needed to support translation
  - Flexibility (sharing, growth, large address space)

Base and bound

- Load each process into a single contiguous region of physical memory
  - Base register: starting physical address
  - Bound register: size of region



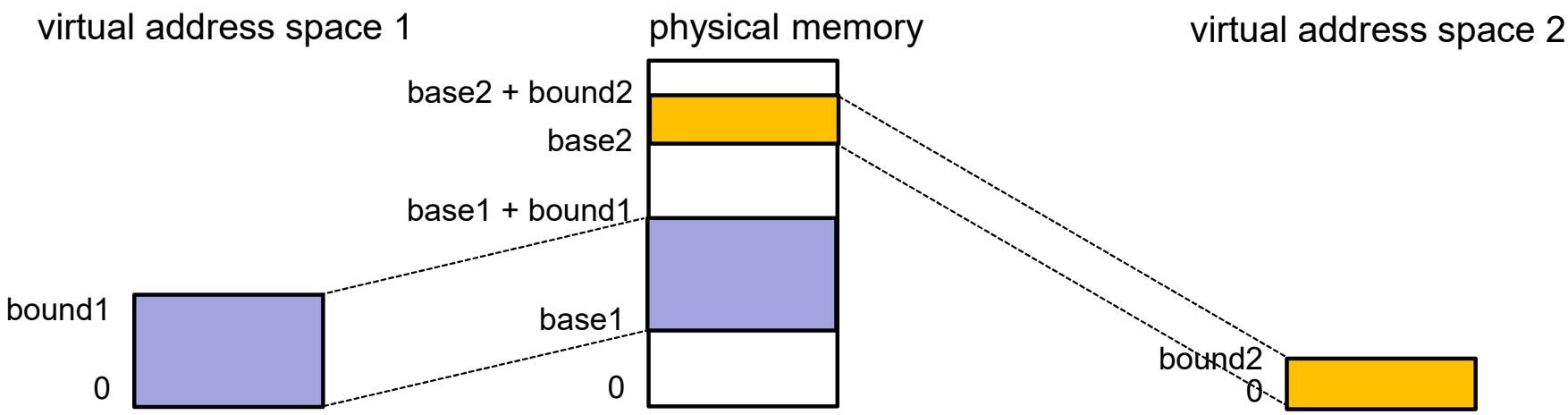
Base and bound: MMU translation algorithm




```
if (virtual address >= bound) {
        trap to kernel; OS kills process (segmentation violation)
                        or retries access
} else {
        physical address = virtual address + base
}
```

How to switch address spaces?

Base and bound: switching address spaces



Base and bound

- Fast
- Simple hardware support

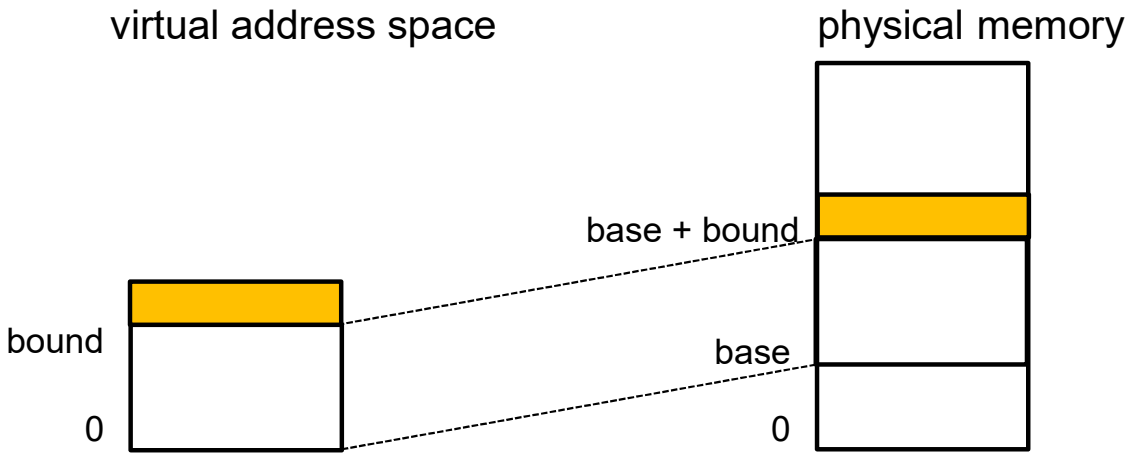
✓ Address independence?

✓ Protection?

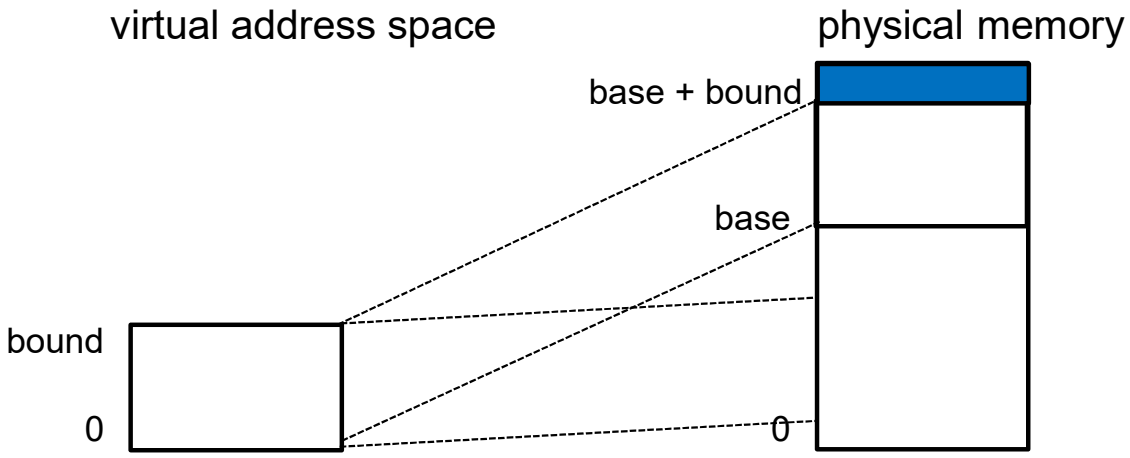
✘ Large address space?

- Flexibility?

Base and bound: how to grow address space?



Base and bound: how to grow address space?



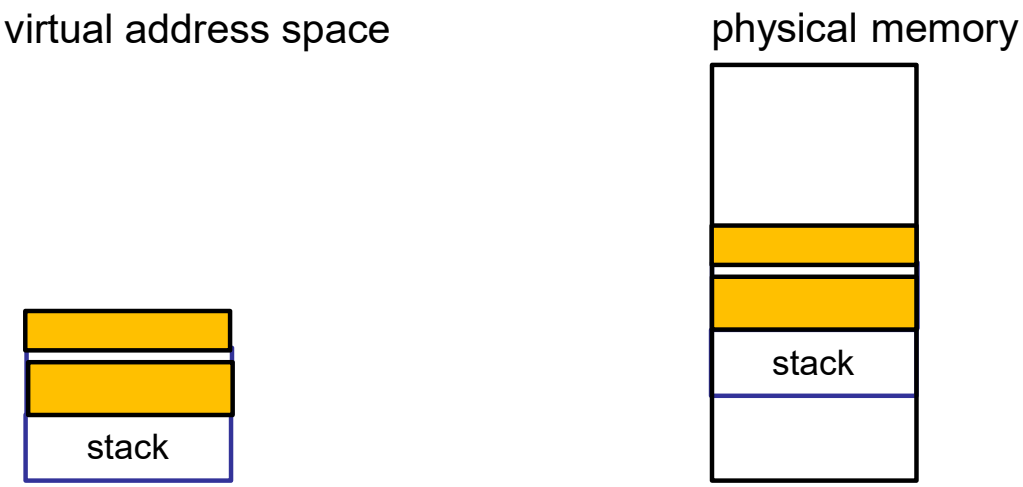
Is process aware of this move?

+ Transparent

- Slow

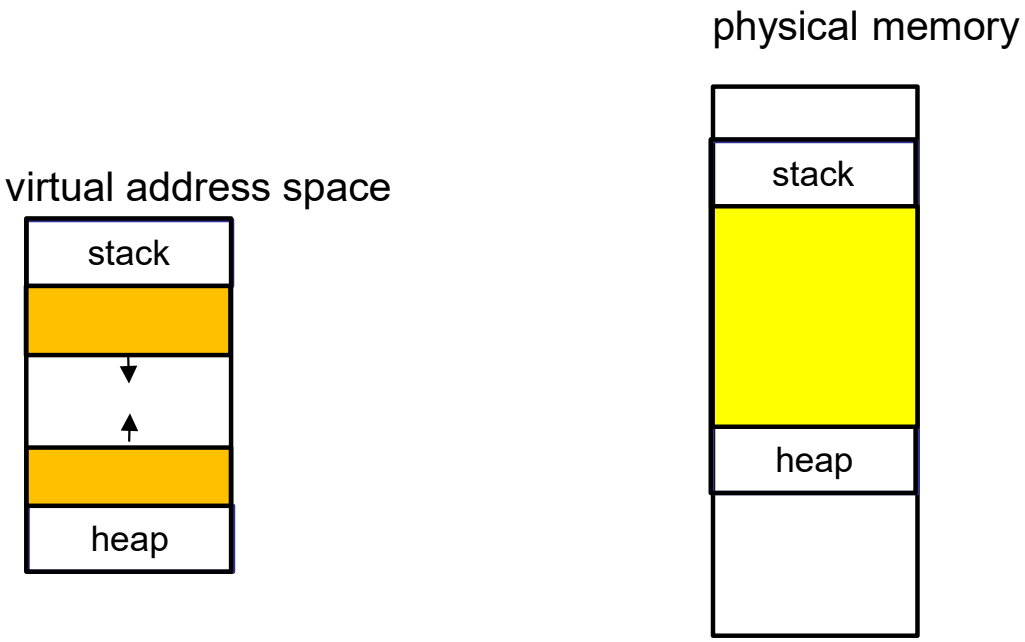
Base and bound: how to grow multiple regions in address space?

- Easy to grow top region (e.g., heap)
- How to grow lower region (e.g., stack)?
  - Problem?



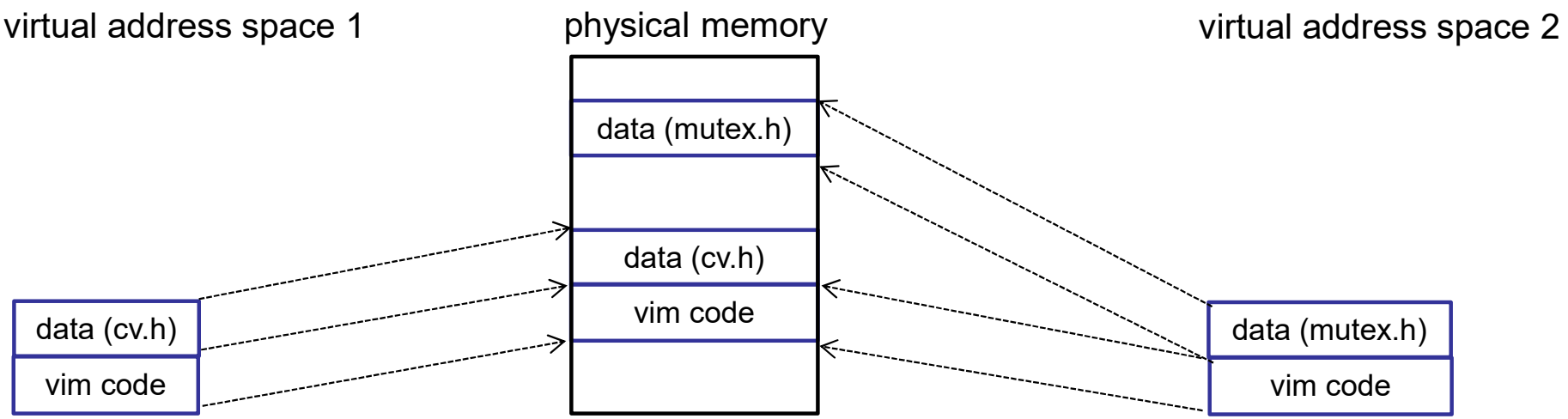
Base and bound: how to grow multiple regions in address space?

- Must reserve space for each region in virtual address space
- Problem?



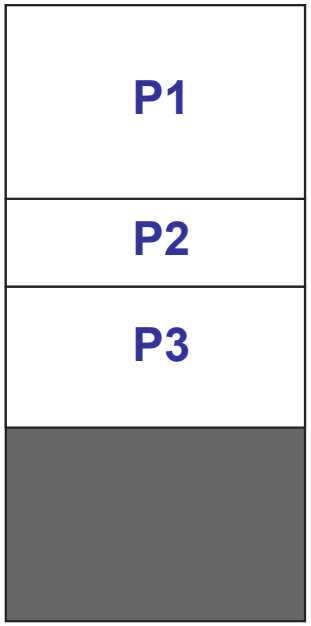
Base and bound: sharing

- Can’t share part of an address space between processes



External fragmentation

- Processes come and go, leaving a mishmash of available memory regions
  - Wasted memory between allocated regions



Base and bound

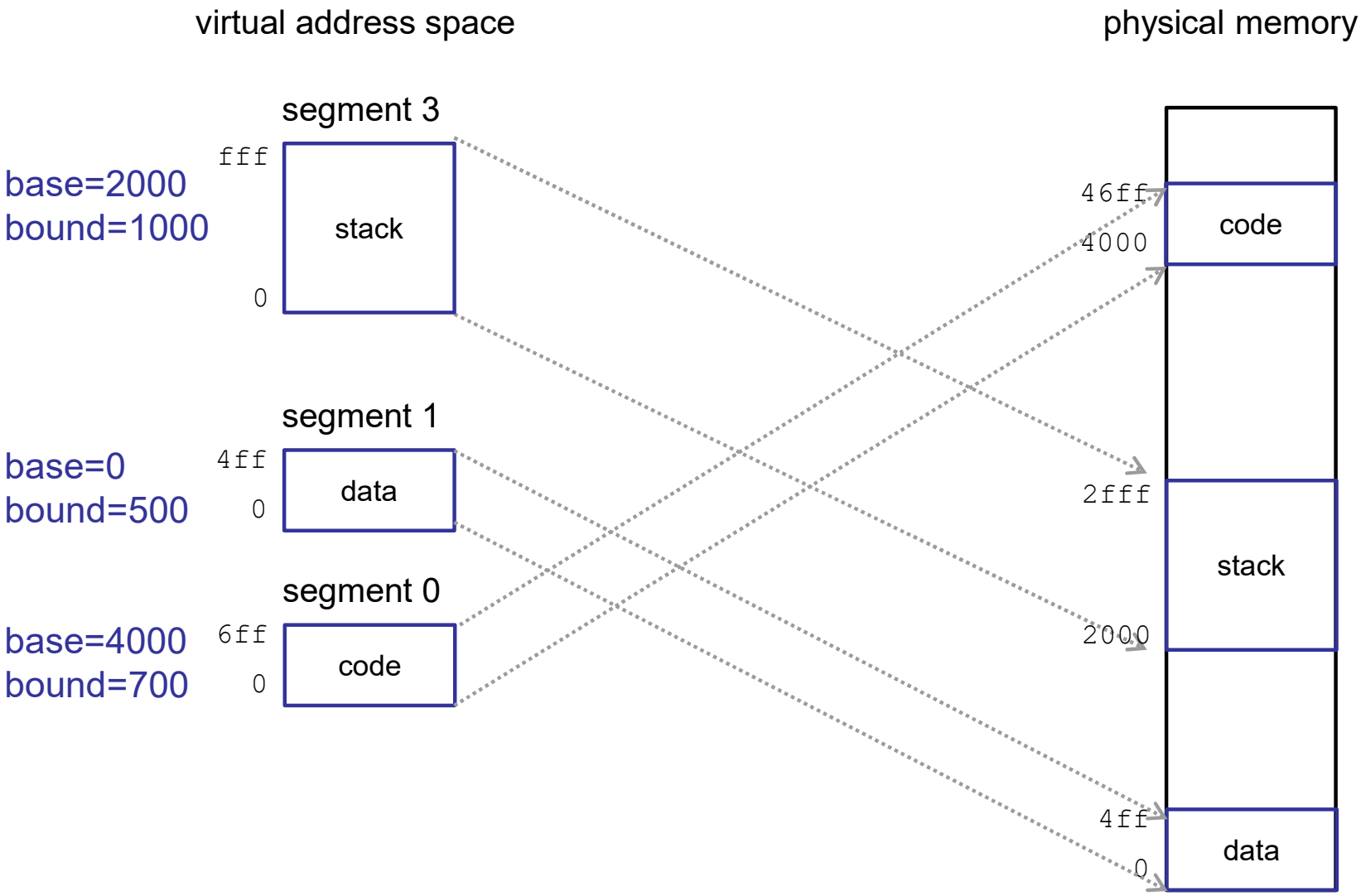
- Pros:
  - Fast
  - Simple hardware support
- Cons:
  - Does not support large address spaces
  - Growing address space may be slow
  - Need to reserve space to allow multiple regions to grow
  - Can’t share part of address space
  - External fragmentation
- Root cause: requires entire address space to be contiguous in memory

Segmentation

- Divide address space into multiple segments

- Segment: region of address space that is:
  - Contiguous in physical memory
  - Contiguous in virtual address space
  - Variable size

Segmentation



Segmentation

Segment #	Base	Bound	Description
3	2000	1000	stack segment
2	n/a	0	unused
1	0	500	data segment
0	4000	700	code segment

- Virtual address: (segment #, offset)
  - Physical address = seg_info[segment #].base + offset
- How to specify the segment #:
  - High bits of address
  - Special register
  - Implicit to instruction opcode

Segmentation: translation

Segment #	Base	Bound	Description
3	2000	1000	stack segment
2	n/a	0	unused
1	0	500	data segment
0	4000	700	code segment

- Physical address for virtual address `(3, 100)` ?
  - `2000 + 100 = 2100`
- Physical address for virtual address `(0, ff)` ?
  - `4000 + ff = 40ff`
- Physical address for virtual address `(1, 2000)` ?
  - invalid (illegal) ➔ segmentation violation/core dump
- Physical address for virtual address `(2, ff)` ?
  - invalid (illegal) ➔ segmentation violation/core dump

Valid vs. invalid addresses

- Not all virtual addresses are valid
  - Valid → address is part of virtual address space
  - Invalid → virtual address is illegal to access
    - » Accessing invalid address causes trap to OS

- In segmentation, a virtual address can be invalid due to:
  - Out of bound offset
  - Invalid segment number

Segmentation: protection

Segment #	Base	Bound	Description	Protection
3	2000	1000	stack segment	read/write
2	n/a	0	unused	
1	0	500	data segment	read/write
0	4000	700	code segment	read

Segmentation: translation algorithm

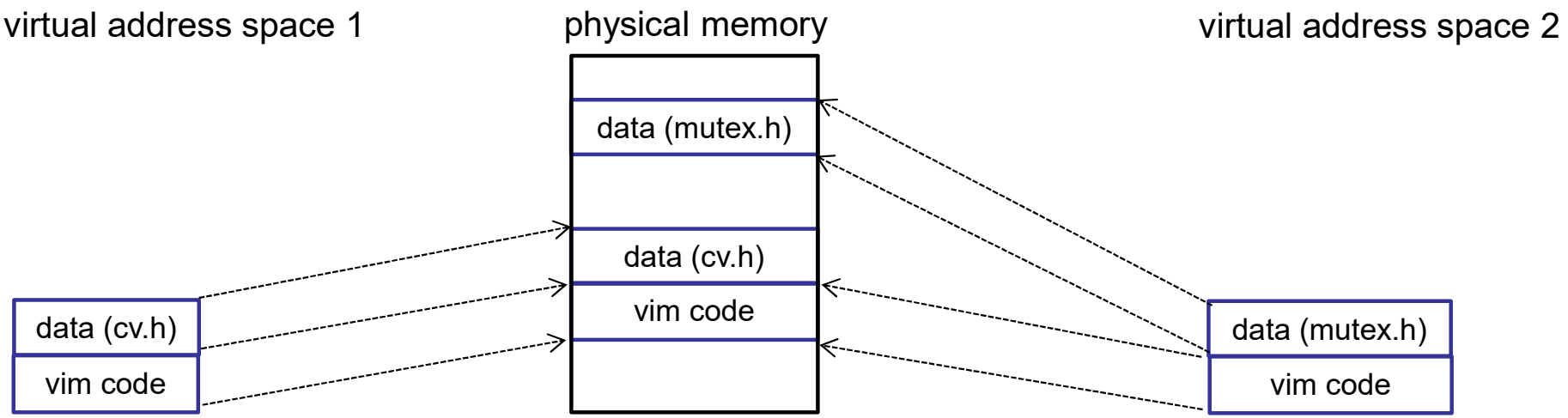
```
(base, bound, protection) = segment_info[segment]

if (segment is invalid or protected, or offset >= bound) {
        trap to kernel
} else {
        physical address = offset + base
}
```

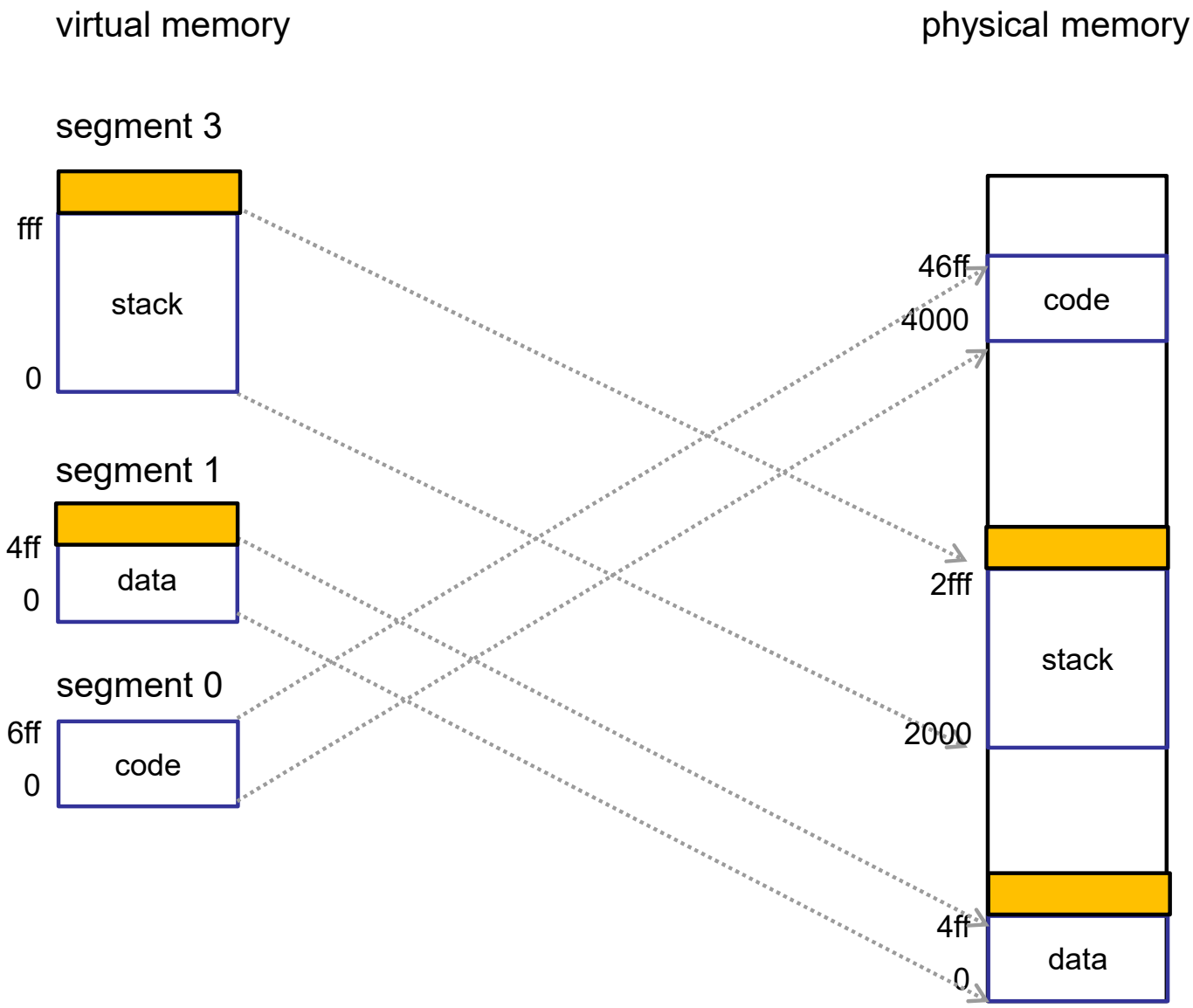
- What must be changed to switch address spaces?

Segment #	Base	Bound	Description	Protection
3	2000	1000	stack segment	read/write
2	n/a	0	unused	
1	0	500	data segment	read/write
0	4000	700	code segment	read

Segmentation supports partial sharing



Segmentation allows multiple growing regions



Segmentation: problems

- Growing address space still slow (may need to copy segment)
- External fragmentation
- Does not support large address spaces?
  - Each segment must fit in memory

- How can we:
  - Make memory allocation easy
  - Not have to worry about external fragmentation
  - Allow address space to be larger than physical memory